

尼恩说在前面

Prometheus圣经的上下两部分

1. prometheus学习圣经（下）：使用prometheus+alertmanager 实现指标预警

2.自定义告警规则

prometheus 告警规则的案例

prometheus告警规则文件的模板化

查看告警规则 and 状态

Prometheus fire 之后

回顾一下：prometheus配置基础规则

3. Alertmanager 的警报管理能力：

3.1 Alertmanager的特性

3.2告警的分组

3.3告警的抑制

3.3告警的静默

4.alertmanager配置及部署

4.1.alertmanager配置

4.2.alertmanager部署

4.3prometheus关联alertmanager

报警通知方式的配置

通知方式一：邮箱报警

通知方式二：使用webhook (java) 回调发短信

5.prometheus+alertmanager 实现监控预警总结

参考文献：

说在最后：有问题找老架构取经

尼恩说在前面

在40岁老架构师 尼恩的**读者交流群**(50+)中，最近有小伙伴拿到了一线互联网企业如得物、阿里、滴滴、极兔、有赞、希音、百度、网易、美团的面试资格，遇到很多很重要监控预警类的面试题：

怎么做 指标的治理？

怎么做监控预警？

很多很多小伙伴遇到这样的难题，但是没有实操过，也了解不全面，导致丢了一些很好的机会。

尼恩结合自己的《10WqpsNetty网关实操操作》，给大家梳理 一本配套的《Prometheus圣经》，帮助大家一招制敌。

同时，把这个指标监控预警相关面试题，也收入咱们的《[尼恩Java面试宝典PDF](#)》V175版本《架构专题》，供后面的小伙伴参考，提升大家的 3高 架构、设计、开发水平。

《尼恩 架构笔记》《尼恩高并发三部曲》《尼恩Java面试宝典》《Prometheus圣经》的PDF，请到文末公号【技术自由圈】获取

Prometheus圣经的上下两部分

- prometheus学习圣经Part1：使用prometheus+Grafana实现指标监控
- prometheus学习圣经Part2：使用prometheus+Alertmanager实现指标预警

Prometheus 学习圣经

尼恩 编著

未来职业, 如何突围: 三栖架构师

FSAC 三栖合一架构师

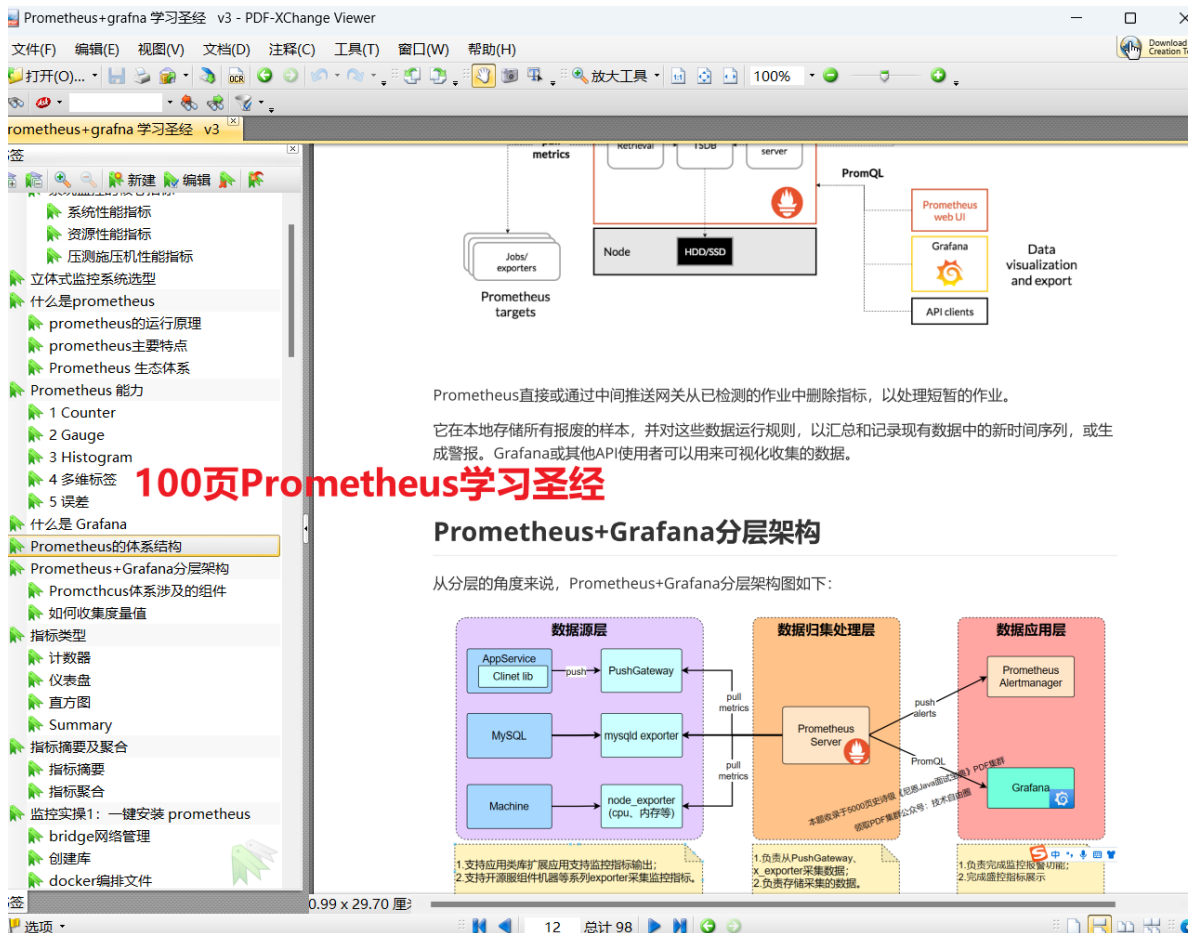
Future Super Architect Community

- **第一栖: Java架构**
- **第二栖: GO架构**
- **第三栖: 大数据架构**

一次穿透GPA底层原理

**疯狂创客圈Java高并发社区
(技术自由圈 内部资料)**

完整的PDF, 请在 技术自由圈 公众号获取。

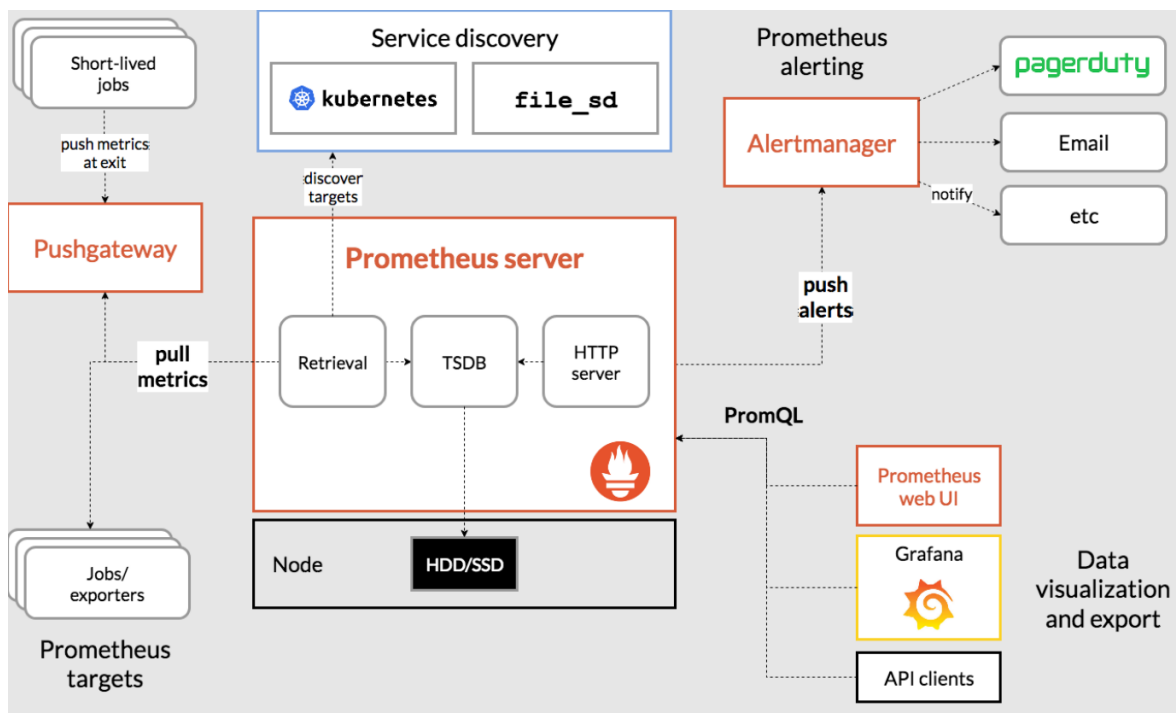


1. prometheus学习圣经（下）：使用 prometheus+alertmanager 实现指标预警

监控和告警，是孪生兄弟，监控就需要报警，没有报警的监控是不完善的。

那么prometheus生态，报警究竟是怎么实现呢，告警组件它又是怎么和prometheus一起协调，并且完成报警的发送呢？

先来看一下，prometheus的官方生态架构图



prometheus可以抓取exporter中暴露出来的指标数据并且存储，prometheus有自己的查询语句PromQL，

grafana面板中的显示的都是PromQL查询prometheus时序数据库出来的数据。

和grafana一样，告警组件alertmanager也是依靠prometheus的PromQL，具体的流程大致是：在prometheus中可以定义一些PromQL语句，当语句查询出来的值超过预定的阈值，prometheus发送报警给alertmanager了，

alertmanager负责把prometheus发送过去的相关的报警指标分类，并且执行报警（微信，邮件，企微）

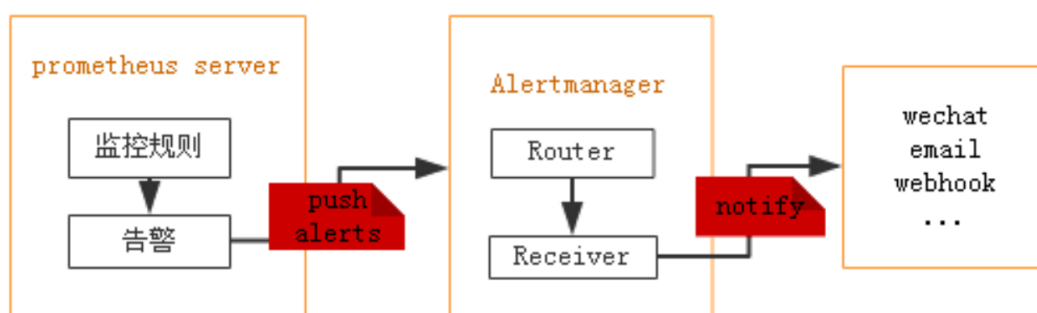
Prometheus的架构中被划分为两个部分，

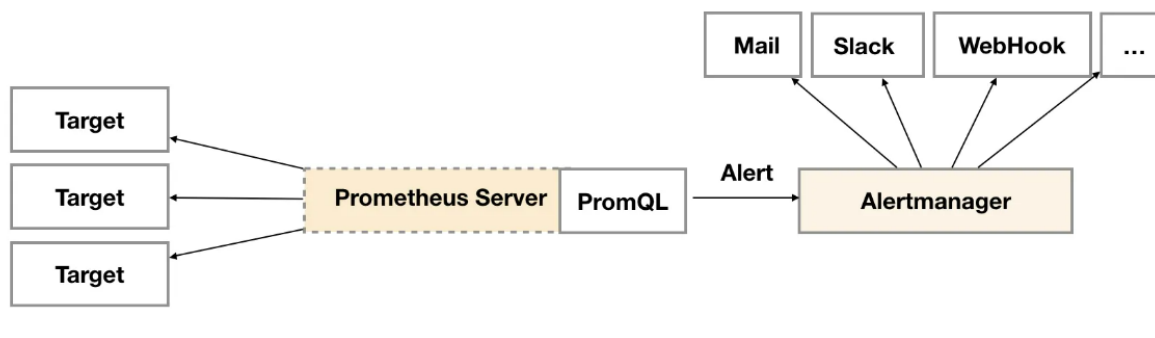
1. 在Prometheus Server中定义告警规则，以及产生告警，
2. Alertmanager组件则用于处理这些由Prometheus产生的告警。

Alertmanager即Prometheus体系中告警的统一处理中心。Alertmanager提供了多种内置第三方告警通知方式，同时还提供了对Webhook通知的支持，通过Webhook用户可以完成对告警更多个性化的扩展。

prometheus触发一条告警的过程：

prometheus—>触发阈值—>超出持续时间—>alertmanager—>分组|抑制|静默—>媒体类型—>邮件|钉钉|微信等。





Alertmanager: 从 Prometheus server 端接收到 alerts 后，会进行去重，分组，并路由到相应的接收方，发出报警，常见的接收方式有：电子邮件，微信，钉钉, slack等。

在Prometheus中一条告警规则主要由以下几部分组成：

1. **告警名称:** 用户需要为告警规则命名，当然对于命名而言，需要能够直接表达出该告警的主要内容
2. **告警规则:** 告警规则实际上主要由PromQL进行定义，其实际意义是当表达式（PromQL）查询结果持续多长时间（During）后出发告警

2.自定义告警规则

Prometheus中的告警规则允许你基于PromQL表达式定义告警触发条件，Prometheus后端对这些触发规则进行周期性计算，当满足触发条件后则会触发告警通知。

默认情况下，用户可以通过Prometheus的Web界面查看这些告警规则以及告警的触发状态。

当Prometheus与Alertmanager关联之后，可以将告警发送到外部服务如Alertmanager中并通过Alertmanager可以对这些告警进行进一步的处理。

prometheus 告警规则的案例

先来一个简单点的案例

```
groups:
- name: example
  rules:
  - alert: HighErrorRate
    expr: job:request_latency_seconds:mean5m{job="myjob"} > 0.5
    for: 10m
    labels:
      severity: page
    annotations:
      summary: High request latency
      description: description info
```

下面是尼恩用来做直接内存阈告警的例子：

```
groups:
# 报警组名称
- name: DirectMemoryAlert
#报警组规则
rules:
#告警名称，需唯一
- alert: DirectMemoryAlert
#promQL表达式
expr: gateway_direct_memory > 5
#满足此表达式持续时间超过for规定的时间才会触发此报警
for: 1m
labels:
#严重级别
severity: page
annotations:
#发出的告警标题
summary: "实例 {{ $labels.instance }} 直接内存使用超过 5M"
#发出的告警内容
description: "实例{{ $labels.instance }} 直接内存使用超过 5M，(当前值为: {{
$value }})M"
```

具体的介绍请参见尼恩配套视频《第35章：中间件塔尖实战，穿透Netty对象池+内存池，直接内存泄露排查》

可以将一组相关的规则设置定义在一个group下。 在每一个group中定义多个告警规则(rule)。

告警规则主要由以下几部分组成：

- `alert`：告警规则的名称。
- `expr`：基于PromQL表达式告警触发条件，用于计算是否有时间序列满足该条件。
- `for`：评估等待时间，可选参数。用于表示只有当触发条件持续一段时间后才发送告警。在等待期间新产生告警的状态为pending。
- `labels`：自定义标签，允许用户指定要附加到告警上的一组附加标签。
- `annotations`：用于指定一组附加信息，比如用于描述告警详细信息的文字等，annotations的内容在告警产生时会一同作为参数发送到Alertmanager。

让Prometheus启用定义的告警规则

在Prometheus全局配置文件中，通过 `rule_files` 指定一组告警规则文件的访问路径，路径下存放**告警规则文件**

Prometheus启动后会自动扫描这些路径下**告警规则文件**中定义的内容，并且根据这些规则计算是否向外部发送通知：

```
rule_files:
[ - <filepath_glob> ... ]
```

默认情况下，Prometheus会每分钟对这些告警规则进行计算，注意周期是 每分钟。

如果用户想 自定义的告警计算周期 ， 则可以通过 `evaluation_interval` 来覆盖默认的计算周期：

```
global:
  [ evaluation_interval: <duration> | default = 1m ]
```

prometheus告警规则文件的模板化

在告警规则文件的annotations中，有两个重要的、关于规则的描述信息项目：

- `summary`：描述告警的概要信息，
- `description`：描述告警的详细信息。

除了Prometheus之外，同时Alertmanager的UI也会根据这两个标签值，显示告警信息。

为了提升可读性，Prometheus支持模板化label和annotations的**中标签的值**，通过表达式去访问。

比如，通过下面两个表达式，可以访问数据序列中的 标签和样本值：

- `$labels.变量`，这个表达式可以访问当前告警实例中指定标签的值。
- `$value`，这个表达式则可以获取当前PromQL表达式计算的样本值。

```
# To insert a firing element's label values:
{{ $labels.<labelname> }}
# To insert the numeric expression value of the firing element:
{{ $value }}
```

举个例子，以通过模板化优化summary以及description的内容的可读性

```
- alert: KubeAPILatencyHigh
  annotations:
    message: The API server has a 99th percentile latency of {{ $value }}
seconds
    for {{ $labels.verb }} {{ $labels.resource }}.
  expr: |

  cluster_quantile:apiserver_request_latencies:histogram_quantile{job="apiserver"
,quantile="0.99",subresource!="log"} > 4
  for: 10m
  labels:
    severity: critical
```

这条警报的**大致**含义是，假如 kube-apiserver 的 P99 响应时间大于 4 秒，并持续 10 分钟以上，就产生报警。

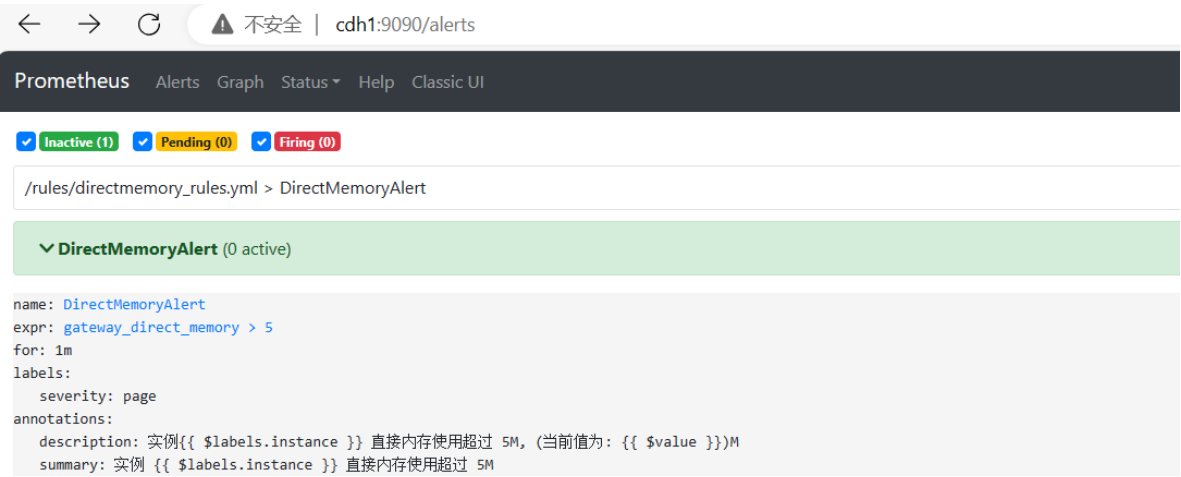
首先要注意的是由 `for` 指定的 Pending Duration（持续时间）。

这个 `for` 参数主要用于降噪，很多类似响应时间这样的指标都是有抖动的，通过指定 Pending Duration，我们可以过滤掉这些瞬时抖动，让 on-call 人员能够把注意力放在真正有持续影响的问题上。

那么显然，下面这样的状况是不会触发这条警报规则的，因为虽然指标已经达到了警报阈值，但持续时间并不够长：

查看告警规则和状态

可以通过Prometheus WEB界面中的Alerts菜单查，看当前Prometheus下的所有告警规则，以及其当前所处的活动状态。



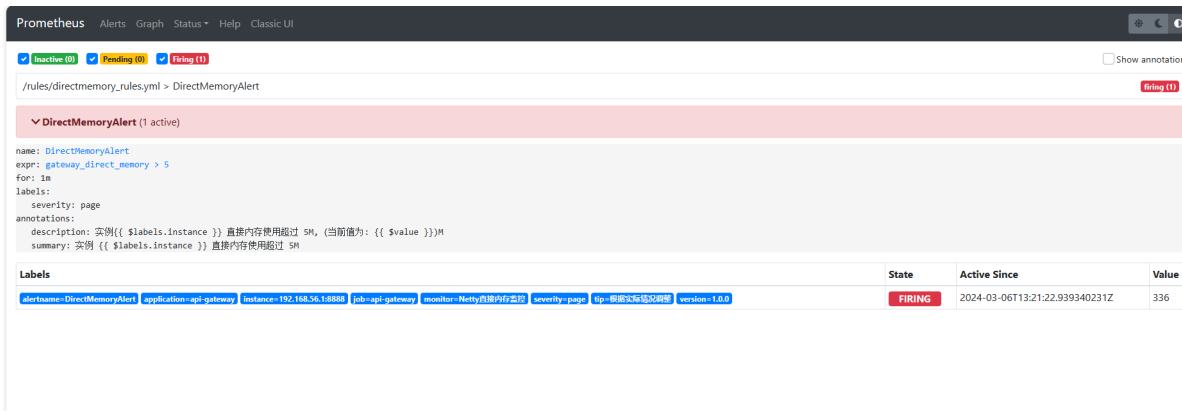
状态说明Prometheus Alert 告警状态有三种状态：Inactive、Pending、Firing。

- 1、Inactive：非活动状态，表示正在监控，但是还未有任何警报触发。
- 2、Pending：表示这个警报必须被触发。由于警报可以被分组、压抑/抑制或静默/静音，所以警报等待验证，一旦所有的验证都通过，则将转到Firing 状态。
- 3、Firing：将警报发送到AlertManager，它将按照配置将警报的发送给所有接收者。一旦警报解除，则将状态转到Inactive，如此循环。

表示这个警报必须被触发。 警报等待验证，告警触发条件满足，但 `for` 定义的持续时间不满足。



Firing：将警报发送到AlertManager，它将按照配置将警报的发送给所有接收者。



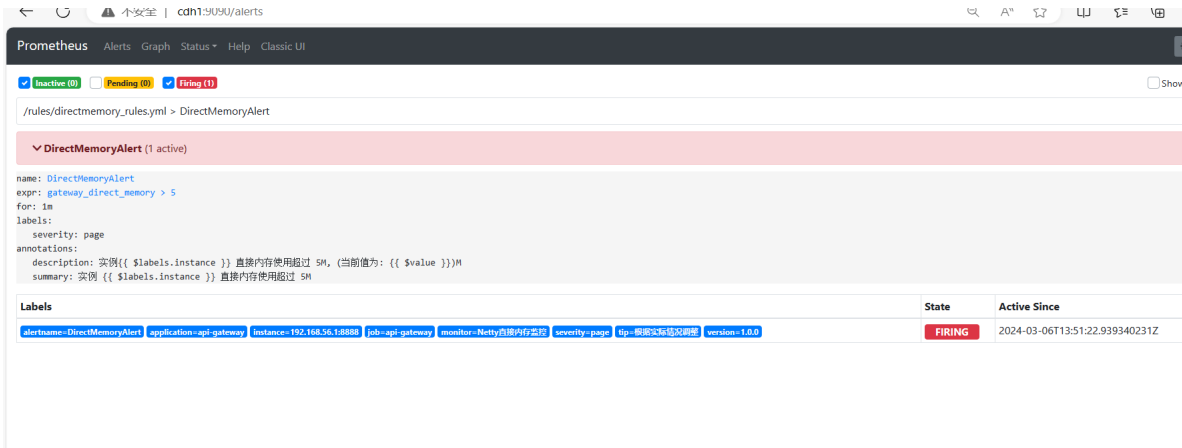
对于已经pending或者firing的告警，Prometheus也会将它们存储到时间序列 `ALERTS{}` 中。 可以通过表达式，查询告警实例：

```
ALERTS{alertname="<alert name>", alertstate="pending|firing", <additional alert labels>}
```

样本值为1表示当前告警处于活动状态（pending或者firing），当告警从活动状态转换为非活动状态时，样本值则为0。

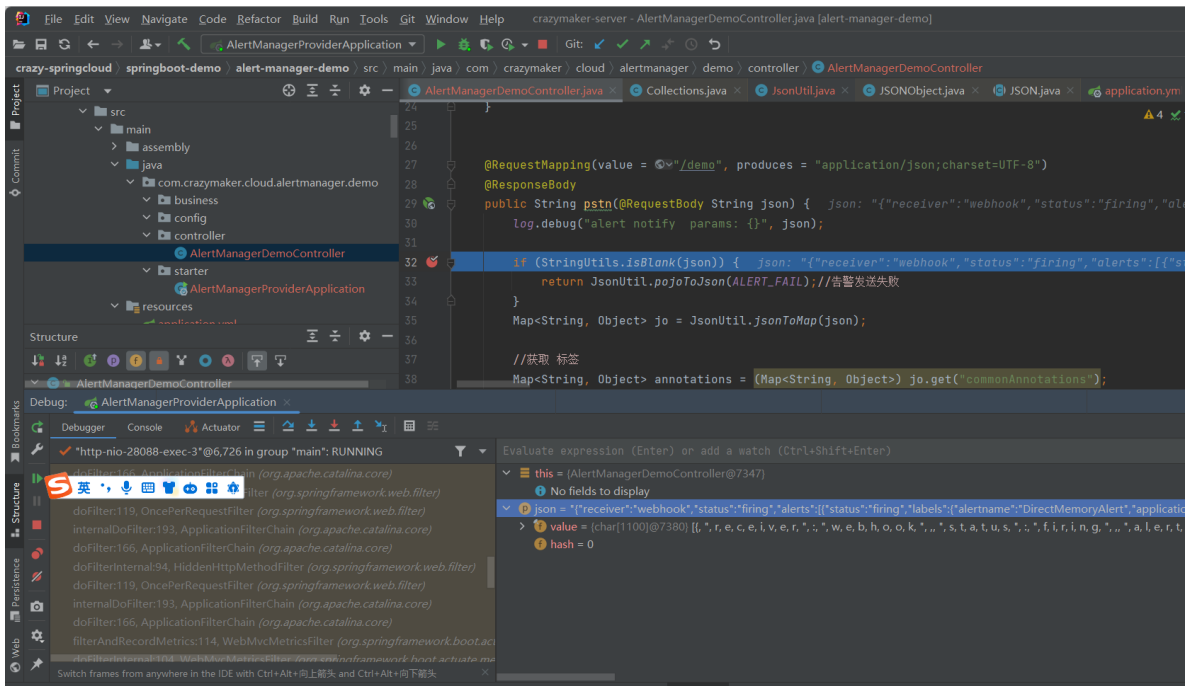
Prometheus fire 之后

Prometheus fire 之后 将警报发送到AlertManager，



AlertManager 将按照配置将警报的发送给所有接收者。

尼恩的直接内存告警案例中，接受者是一个 webhook，收到告警的时候，断点被命中。



回顾一下：prometheus配置基础规则

prometheus.yml 属性配置

属性	描述
scrape_interval	样本采集周期，默认为1分钟采集一次
evaluation_interval	告警规则计算周期，默认为1分钟计算一次
rule_files	指定告警规则的文件
scrape_configs	job的配置项，里面可配多组job任务
job_name	任务名称，需要唯一性
static_configs	job_name 的配置选项，一般使用file_sd_configs 热加载配置
file_sd_configs	job_name 的动态配置选项，使用此配置可以实现配置文件的 热加载
files:	file_sd_configs 配置的服务发现的文件路径列表，支持.json, .yaml 或.yaml, 路径最后一层支持通配符*
refresh_interval	file_sd_configs 中的 files 重新加载的周期，默认5分钟

下面是一个完整的参考案例：

```
# my global config
global:
  scrape_interval:      15s # 采样周期
  evaluation_interval: 15s # 告警规则计算周期
```

```

# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
        - targets:
            - alertmanager:9093

# 报警规则文件可以指定多个，并且可以使用通配符*
rule_files:
  - "rules/*_rules.yml"
  # - "second_rules.yml"

# 采集job配置
scrape_configs:
  - job_name: 'prometheus'
    #使用file_sd_configs热加载配置文件
    file_sd_configs:
      #指定1分钟加载一次配置
      - refresh_interval: 1m
        files:
          - config_prometheus.json

  - job_name: 'linux_base'
    file_sd_configs:
      - refresh_interval: 1m
        files:
          - config_exporter.json

  - job_name: 'service_base'
    file_sd_configs:
      - refresh_interval: 1m
        files:
          - config_mtail.json

  - job_name: 'grafana'
    file_sd_configs:
      - refresh_interval: 1m
        files:
          - config_grafana.json

```

具体的介绍请参见尼恩配套视频《第35章：中间件塔尖实战，穿透Netty对象池+内存池，直接内存泄露排查》

此处我们使用 `rule_files` 属性来设置告警文件，这里设置报警规则，rules/host_rules.yml

```

groups:
# 报警组名称
- name: hostStatsAlert
#报警组规则
rules:
#告警名称，需唯一
- alert: hostCpuUsageAlert

```

```

#promQL表达式
expr: sum(avg without (cpu)(irate(node_cpu_seconds_total{mode!='idle'}
[5m]))) by (instance) > 0.85
#满足此表达式持续时间超过for规定的时间才会触发此报警
for: 1m
labels:
  #严重级别
  severity: page
annotations:
  #发出的告警标题
  summary: "实例 {{ $labels.instance }} CPU 使用率过高"
  #发出的告警内容
  description: "实例{{ $labels.instance }} CPU 使用率超过 85% (当前值为: {{
$value }})"
- alert: hostMemUsageAlert
  expr: (node_memory_MemTotal_bytes -
node_memory_MemAvailable_bytes)/node_memory_MemTotal_bytes > 0.85
  for: 1m
  labels:
    severity: page
  annotations:
    summary: "实例 {{ $labels.instance }} 内存使用率过高"
    description: "实例 {{ $labels.instance }} 内存使用率 85% (当前值为: {{ $value
}})"

```

下面是尼恩用来做直接内存阈值监控的例子 directmemory_rules.yml

```

groups:
# 报警组名称
- name: DirectMemoryAlert
  #报警组规则
  rules:
    #告警名称, 需唯一
    - alert: DirectMemoryAlert
      #promQL表达式
      expr: gateway_direct_memory > 5
      #满足此表达式持续时间超过for规定的时间才会触发此报警
      for: 1m
      labels:
        #严重级别
        severity: page
      annotations:
        #发出的告警标题
        summary: "实例 {{ $labels.instance }} 直接内存使用超过 5M"
        #发出的告警内容
        description: "实例{{ $labels.instance }} 直接内存使用超过 5M, (当前值为: {{
$value }})M"

```

具体的介绍请参见尼恩配套视频《第35章：中间件塔尖实战，穿透Netty对象池+内存池，直接内存泄露排查》

3. Alertmanager 的警报管理能力：

- Prometheus负责中配置警报规则，将警报发送到Alertmanager。
- Alertmanager 负责管理这些警报，包括沉默，抑制，合并和发送通知。

Alertmanager 发送通知有多种方式，其内部集成了邮箱、Slack、企业微信、webhook等方式，网上也有大量例子实现对第三方软件的集成，如钉钉等。

本文介绍邮件报警方式和通过使用java来搭建webhook自定义通知报警的方式。

3.1 Alertmanager的特性

Alertmanager除了提供基本的告警通知能力以外，还主要提供了如：分组、抑制以及静默等告警特性



3.2告警的分组

分组(group): 将类似性质的警报合并为单个通知。

在某些情况下，比如由于系统宕机导致大量的告警被同时触发，在这种情况下分组机制可以将这些被触发的告警合并为一个告警通知。

告警的分组 避免一次性接受大量的告警通知，而无法对问题进行快速定位。

例如，当集群中有数百个正在运行的服务实例，并且为每一个实例设置了告警规则。

假如此时发生了网络故障，可能导致大量的服务实例无法连接到数据库，结果就会有数百个告警被发送到Alertmanager。

而作为用户，可能只希望能够在在一个通知中就能查看哪些服务实例受到影响。

这时可以按照服务所在集群或者告警名称对告警进行分组，而将这些告警内聚在一起成为一个通知。

告警分组，告警时间，以及告警的接受方式可以通过Alertmanager的配置文件进行配置。

3.3告警的抑制

抑制(inhibition): 当警报发出后，停止重复发送由此警报引发的其他警报，即合并一个故障引起的多个报警事件，可以消除冗余告警

例如，当集群不可访问时触发了一次告警，通过配置Alertmanager可以忽略与该集群有关的其它所有告警。这样可以避免接收到大量与实际无关的告警通知。

抑制机制同样通过Alertmanager的配置文件进行设置。

3.3告警的静默

静默(silences): 是一种简单的特定时间静音的机制,

例如: 服务器要升级维护可以先设置这个时间段告警静默。

静默提供了一个简单的机制, 可以快速根据标签对告警进行静默处理。

如果接收到的告警符合静默的配置, Alertmanager则不会发送告警通知。

静默设置需要在Alertmanager的Web页面上进行设置。

4.alertmanager配置及部署

4.1.alertmanager配置

alertmanager会定义一个基于标签匹配规则的路由树, 用以接收到报警后根据不同的标签匹配不同的路由, 来将报警发送给不同的receiver。

```
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname']
  group_wait: 10s
  group_interval: 1m
  repeat_interval: 4h
  receiver: 'webhook'
receivers:
- name: 'webhook'
  webhook_configs:
    - url: 'http://192.168.56.1:28088/alert-demo/hook/demo'
```

具体的介绍请参见尼恩配套视频 《第35章: 中间件塔尖实战, 穿透Netty对象池+内存池, 直接内存泄露排查》

如果只是定义一个路由route, 则这个路由需要接收并处理所有的报警,

如果需要区分不同的报警发送给不同的receiver, 则需要配置多个子级route来处理不同的报警, 而根route此时必须能够接收处理所有的报警。

下面是来自于互联网的其他小伙伴的, 一个多个子级route来处理不同的报警的例子

```
#根路由
route:
  #顶级路由配置的接收者(匹配不到子级路由, 会使用根路由发送报警)
  receiver: 'default-receiver'
  #设置等待时间, 在此等待时间内如果接收到多个报警, 则会合并成一个通知发送给receiver
  group_wait: 30s
  #两次报警通知的时间间隔, 如: 5m, 表示发送报警通知后, 如果5分钟内再次接收到报警则不会发送通知
  group_interval: 5m
  #发送相同告警的时间间隔, 如: 4h, 表示4小时内不会发送相同的报警
  repeat_interval: 4h
  #分组规则, 如果满足group_by中包含的标签, 则这些报警会合并为一个通知发给receiver
```

```

group_by: [cluster, alertname]
routes:
  #子路由的接收者
  - receiver: 'database-pager'
    group_wait: 10s
    #默认为false。false: 配置到满足条件的子节点点后直接返回, true: 匹配到子节点后还会继续遍历
    #后续子节点
    continue:false
    #正则匹配, 验证当前标签service的值是否满足当前正则的条件
    match_re:
      service: mysql|cassandra
  #子路由的接收者
  - receiver: 'frontend-pager'
    group_by: [product, environment]
    #字符串匹配, 匹配当前标签team的值为frontend的报警
    match:
      team: frontend

#定义所有接收者
receivers:
  #接收者名称
  - name: 'default-receiver'
    #接收者为webhook类型
    webhook_configs:
      #webhook的接收地址
      - url: 'http://127.0.0.1:5001/'
  - name: 'database-pager'
    webhook_configs:
      - url: 'http://127.0.0.1:5002/'
  - name: 'frontend-pager'
    webhook_configs:
      - url: 'http://127.0.0.1:5003/'

```

4.2.alertmanager部署

使用尼恩的一键部署脚本, 就可以了

```

version: "3.5"
services:
  prometheus:
    image: prom/prometheus:v2.30.0
    container_name: prometheus
    volumes:
      - /tmp:/tmp
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
      - ./rules:/rules
    command: "--config.file=/etc/prometheus/prometheus.yml --
storage.tsdb.path=/prometheus --storage.tsdb.wal-compression --
storage.tsdb.retention.time=7d"
    hostname: prometheus
    environment:
      TZ: "Asia/Shanghai"
    ports:
      - "9090:9090"
    networks:
      base-env-network:

```

```
aliases:
  - prometheus

alertmanager:
  image: prom/alertmanager:v0.23.0
  # restart: always
  container_name: alertmanager
  hostname: alertmanager
  environment:
    TZ: "Asia/Shanghai"
  ports:
    - 9093:9093
  command:
    - "--config.file=/etc/alertmanager/alertmanager.yml"
    - "--log.level=debug"
  volumes:
    - ./alertmanager.yml:/etc/alertmanager/alertmanager.yml
    - ./alertmanager:/alertmanager
```

具体的介绍请参见尼恩配套视频 《第35章：中间件塔尖实战，穿透Netty对象池+内存池，直接内存泄露排查》

其中涉及到直接内存的告警配置，修改alertmanager.yml，内容如下：

```
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname']
  group_wait: 10s
  group_interval: 1m
  repeat_interval: 4h
  receiver: 'webhook'
receivers:
- name: 'webhook'
  webhook_configs:
    - url: 'http://192.168.56.1:28088/alert-demo/hook/demo'
```

启动后访问 <http://cdh1:9093/> 查看alertmanager 是否启动成功，

可以通过 status 按钮，查看alertmanager及其配置：

<http://cdh1:9093/#/alerts>

BuildDate: 20210825-10:48:55
BuildUser: root@e21a959be8d2
GoVersion: go1.16.7
Revision: 61046b17771a57cfd4c4a51be370ab930a4d7d54
Version: 0.23.0

Config

```
global:
  resolve_timeout: 5m
  http_config:
    follow_redirects: true
  smtp_hello: localhost
  smtp_require_tls: true
  pagerduty_url: https://events.pagerduty.com/v2/enqueue
  opsgenie_api_url: https://api.opsgenie.com/
  wechat_api_url: https://qyapi.weixin.qq.com/cgi-bin/
  victorops_api_url: https://alert.victorops.com/integrations/generic/20131114/alert/
route:
  receiver: webhook
  group_by:
    - alertname
  continue: false
  group_wait: 10s
  group_interval: 1m
  repeat_interval: 4h
receivers:
  - name: webhook
    webhook_configs:
      - send_resolved: true
        http_config:
          follow_redirects: true
          url: http://192.168.56.1:28088/alert-demo/hook/demo
        max_alerts: 0
    templates: []
```

4.3prometheus关联alertmanager

prometheus.yml中的alerting标签下配置上alertmanager的地址即可，配置如下：

```
# Alertmanager configuration
alerting:
  alertmanagers:
    - static_configs:
      - targets: ['192.168.199.23:9093']
```

添加后重启prometheus即可。

报警通知方式的配置

通知方式一：邮箱报警

一个简单的 来自互联网的demo，大致如下：

```
global:
  #超时时间
  resolve_timeout: 5m
  #smtp地址需要加端口
  smtp_smarthost: 'smtp.126.com:25'
  smtp_from: 'xxx@126.com'
  #发件人邮箱账号
```

```
smtp_auth_username: 'xxx@126.com'
#账号对应的授权码（不是密码），阿里云个人版邮箱目前好像没有授权码，126邮箱授权码可以在“设置”里面找到
smtp_auth_password: '1qaz2wsx'
smtp_require_tls: false

route:
  group_by: ['alertname']
  group_wait: 10s
  group_interval: 1m
  repeat_interval: 4h
  receiver: 'mail'
receivers:
- name: 'mail'
  email_configs:
  - to: 'xxx@aliyun.com'
```

设置后如果有通知，即可收到邮件如下：



先来看一下alertmanager的配置文件结构

```
global:
  smtp_smarthost: 'smtp.qq.com:465'
  smtp_from: 'xxxxxx@qq.com'
  smtp_auth_username: 'xxxxxxxxx@qq.com'
  smtp_auth_password: 'xxxxxxxxx'
  smtp_require_tls: false
route:
  group_by: [alertname]
  group_wait: 5s
  group_interval: 5m
  repeat_interval: 5s
  receiver: default-receiver
receivers:
- name: 'default-receiver'
  email_configs:
  - to: 'xxxx@163.com'
```

通知方式二：使用webhook (java) 回调发短信

```
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname']
  group_wait: 10s
  group_interval: 1m
  repeat_interval: 4h
  receiver: 'webhook'
receivers:
- name: 'webhook'
  webhook_configs:
  - url: 'http://192.168.56.1:28088/alert-demo/hook/demo'
```

使用webhook方式，alertmanager调用webhook钩子地址，发送post请求，
Post请求参数为json字符串（字符串类型）：

```
{
  "receiver": "webhook",
  "status": "firing",
  "alerts": [
    {
      "status": "firing",
      "labels": {
        "alertname": "DirectMemoryAlert",
        "application": "api-gateway",
        "instance": "192.168.56.1:8888",
        "job": "api-gateway",
        "monitor": "Netty直接内存监控",
        "severity": "page",
        "tip": "根据实际情况调整",
        "version": "1.0.0"
      },
      "annotations": {
        "description": "实例192.168.56.1:8888 直接内存使用超过 5M，(当前值为: 288)M",
        "summary": "实例 192.168.56.1:8888 直接内存使用超过 5M"
      },
      "startsAt": "2024-03-11T05:51:22.939Z",
      "endsAt": "0001-01-01T00:00:00Z",
      "generatorURL": "http://prometheus:9090/graph?g0.expr=gateway_direct_memory+%3E+5\u0026g0.tab=1",
      "fingerprint": "2e3375fead9bb5fe"
    }
  ],
  "groupLabels": {
    "alertname": "DirectMemoryAlert"
  },
  "commonLabels": {
```

```

    "alertname": "DirectMemoryAlert",
    "application": "api-gateway",
    "instance": "192.168.56.1:8888",
    "job": "api-gateway",
    "monitor": "Netty直接内存监控",
    "severity": "page",
    "tip": "根据实际情况调整",
    "version": "1.0.0"
  },
  "commonAnnotations": {
    "description": "实例192.168.56.1:8888 直接内存使用超过 5M, (当前值为: 288)M",
    "summary": "实例 192.168.56.1:8888 直接内存使用超过 5M"
  },
  "externalURL": "http://alertmanager:9093",
  "version": "4",
  "groupKey": "{}:{alertname=\"DirectMemoryAlert\"}",
  "truncatedAlerts": 0
}

```

此时需要使用java（其他任何语言都可以，反正只要能处理http的请求就行）搭建个http的请求处理器来处理报警通知，如下：

```

package com.crazymaker.cloud.alertmanager.demo.controller;

import com.crazymaker.springcloud.common.util.JsonUtil;
import com.crazymaker.springcloud.common.util.StringUtils;
import lombok.extern.slf4j.Slf4j;
import org.springframework.web.bind.annotation.*;

import java.util.*;

@RestController
@RequestMapping("/hook")
@Slf4j
public class AlertManagerDemoController {

    public static final Map<String, Object> ALERT_FAIL = new HashMap<>();
    public static final Map<String, Object> ALERT_SUCCESS = new HashMap<>();

    static {
        ALERT_FAIL.put("msg", "报警失败");
        ALERT_FAIL.put("code", 0);
        ALERT_SUCCESS.put("msg", "报警成功");
        ALERT_SUCCESS.put("code", 1);
    }

    @RequestMapping(value = "/demo", produces = "application/json;charset=UTF-8")
    @ResponseBody
    public String pstn(@RequestBody String json) {
        log.debug("alert notify params: {}", json);

        if (StringUtils.isBlank(json)) {
            return JsonUtil.pojoToJson(ALERT_FAIL); //告警发送失败

```

```

    }
    Map<String, Object> jo = JsonUtil.jsonToMap(json);

    //获取 标签
    Map<String, Object> annotations = (Map<String, Object>)
jo.get("commonAnnotations");

    if (annotations == null) {
        return JsonUtil.pojoToJson(ALERT_FAIL); //告警发送失败
    }

    Object status = jo.get("status");
    Object subject = annotations.get("summary");
    Object content = annotations.get("description");
    log.error("status: {}", status);
    log.error("summary: {}", subject);
    log.error("content: {}", content);

    //开发发送短信
    List<String> users = getRelatedPerson();

    try {
        boolean success = sendMsg(subject, content, users);
        if (success) {
            return JsonUtil.pojoToJson(ALERT_SUCCESS); //告警发送成功
        }
    } catch (Exception e) {
        log.error("=alert sms notify error. json={}", json, e);
    }
    return JsonUtil.pojoToJson(ALERT_FAIL); //告警发送失败
}

//todo 发送短信

private boolean sendMsg(Object subject, Object content, List<String> users)
{
    return true;
}

//todo 从配置中心，加载短信号码
private List<String> getRelatedPerson() {
    return Collections.emptyList();
}
}

```

具体的介绍请参见尼恩配套视频《第35章：中间件塔尖实战，穿透Netty对象池+内存池，直接内存泄露排查》

```
020 x86_64 prometheus (none))"
prometheus | level=info ts=2024-03-06T13:44:56.293Z caller=main.go:445 fd_limits="(soft=1048576, hard=1048576)"
prometheus | level=info ts=2024-03-06T13:44:56.293Z caller=main.go:446 vm_limits="(soft=unlimited, hard=unlimited)"
prometheus | level=info ts=2024-03-06T13:44:56.295Z caller=web.go:541 component=web msg="Start listening for connections" address=0.0.0.0:9090
prometheus | level=info ts=2024-03-06T13:44:56.295Z caller=main.go:822 msg="Starting TSDB ..."
prometheus | level=info ts=2024-03-06T13:44:56.296Z caller=tsdb/config.go:101 component=web msg="TLS is disabled." http2=false
prometheus | level=info ts=2024-03-06T13:44:56.297Z caller=head.go:466 component=tsdb msg="Replaying on-disk memory mappable chunks if any"
prometheus | level=info ts=2024-03-06T13:44:56.297Z caller=head.go:500 component=tsdb msg="On-disk memory mappable chunks replay completed" duration=2.663µs
prometheus | level=info ts=2024-03-06T13:44:56.297Z caller=head.go:506 component=tsdb msg="Replaying WAL, this may take a while"
prometheus | level=info ts=2024-03-06T13:44:56.297Z caller=head.go:577 component=tsdb msg="WAL segment loaded" segment=0 maxSegment=0
prometheus | level=info ts=2024-03-06T13:44:56.297Z caller=head.go:583 component=tsdb msg="WAL replay completed" checkpoint_replay_duration=22.272µs wal_replay_duration=133.711µs total_replay_duration=172.617µs
prometheus | level=info ts=2024-03-06T13:44:56.298Z caller=main.go:849 fs_type=XFS_SUPER_MAGIC
prometheus | level=info ts=2024-03-06T13:44:56.298Z caller=main.go:852 msg="TSDB started"
prometheus | level=info ts=2024-03-06T13:44:56.298Z caller=main.go:979 msg="Loading configuration file" filename=/etc/prometheus/prometheus.yml
prometheus | level=info ts=2024-03-06T13:44:56.299Z caller=main.go:1016 msg="Completed loading of configuration file" filename=/etc/prometheus/prometheus.yml totalDuration=1.023436ms db_storage=1.406µs remote_storage=2.018µs web_handler=137ns query_engine=1.723µs scrape=182.759µs scrape_sd=53.917µs notify=19.598µs notify_sd=9.717µs rules=507.817µs
prometheus | level=info ts=2024-03-06T13:44:56.299Z caller=main.go:794 msg="Server is ready to receive web requests."
prometheus | level=info ts=2024-03-06T13:44:58.227Z caller=cluster.go:696 component=cluster msg="gossip not settled" polls=0 before=0 now=1 elapsed=2.005152761s
alertmanager | level=debug ts=2024-03-06T13:45:00.227Z caller=cluster.go:693 component=cluster msg="gossip looks settled" elapsed=4.005823573s
alertmanager | level=debug ts=2024-03-06T13:45:00.232Z caller=cluster.go:693 component=cluster msg="gossip looks settled" elapsed=6.010023457s
alertmanager | level=debug ts=2024-03-06T13:45:04.235Z caller=cluster.go:693 component=cluster msg="gossip looks settled" elapsed=8.013178259s
alertmanager | level=info ts=2024-03-06T13:45:06.235Z caller=cluster.go:688 component=cluster msg="gossip settled; proceeding" elapsed=10.013408111s
alertmanager | level=debug ts=2024-03-06T13:59:56.221Z caller=flog.go:336 component=flog msg="Running maintenance"
alertmanager | level=debug ts=2024-03-06T13:59:56.221Z caller=silence.go:360 component=silences msg="Running maintenance"
alertmanager | level=debug ts=2024-03-06T13:59:56.232Z caller=flog.go:338 component=flog msg="Maintenance done" duration=10.769943ms size=0
alertmanager | level=debug ts=2024-03-06T13:59:56.232Z caller=silence.go:362 component=silences msg="Maintenance done" duration=10.924046ms size=0
alertmanager | level=debug ts=2024-03-06T14:01:22.944Z caller=dispatch.go:165 component=dispatcher msg="Received alert" alert=DirectMemoryAlert[2e3375f][active]
alertmanager | level=debug ts=2024-03-06T14:01:32.945Z caller=dispatch.go:516 component=dispatcher aggGroup="{}:{alertname=\"DirectMemoryAlert\"}" msg=flusing alert=DirectMemoryAlert[2e3375f][active]
alertmanager | level=error ts=2024-03-06T14:02:32.947Z caller=dispatch.go:354 component=dispatcher msg="Notify for alerts failed" num_alerts=1 err="webhook /webhook[0]: notify retry canceled after 2 attempts: Post \"http://192.168.56.1:28088/alert-demo/hook/demo\": context deadline exceeded"
alertmanager | level=debug ts=2024-03-06T14:02:32.948Z caller=dispatch.go:516 component=dispatcher aggGroup="{}:{alertname=\"DirectMemoryAlert\"}" msg=flusing alert=DirectMemoryAlert[2e3375f][active]
alertmanager | level=debug ts=2024-03-06T14:02:53.472Z caller=notify.go:735 component=dispatcher receiver=webhook integration=webhook[0] msg="Notify success" attempts=1
alertmanager | level=debug ts=2024-03-06T14:03:32.953Z caller=dispatch.go:516 component=dispatcher aggGroup="{}:{alertname=\"DirectMemoryAlert\"}" msg=flusing alert=DirectMemoryAlert[2e3375f][active]
```

5.prometheus+alertmanager 实现监控预警总结

回顾一下，时间相关参数

参数名称	说明	默认值	参数所属
scrape_interval	指标数据采集间隔	1分钟	prometheus.yml
evaluation_interval	规则的计算间隔	1分钟	prometheus.yml
for: 时间	异常持续多长时间发送告警	0	规则配置
group_wait	分组等待时间。同一分组内收到第一个告警等待多久开始发送，目的是为了同组消息同时发送	30秒	alertmanager.yml
group_interval	上下两组发送告警的间隔时间。第一次告警发出后等待group_interval时间，开始为该组触发新告警	5分钟	alertmanager.yml
repeat_interval	重发间隔。告警已经发送，且无新增告警，再次发送告警需要的间隔时间	4小时	alertmanager.yml

alertmanager配置功能确实很强大，完全能够按照自己的目标定义灵活的通知和报警方式，尤其是对webhook的支持，简直不能更灵活了。

而且alertmanager还支持定义template来更清晰的彰显要通知的内容。

但是本人还没有发现alertmanager该怎么进行配置的热加载，后面还要再研究下怎么优化通知，避免告警轰炸和提高告警的准确性。

参考文献：

<https://prometheus.io/download/>

说在最后：有问题找老架构取经

怎么做 指标的治理？

怎么做监控预警？

以上的内容，如果大家能对答如流，如数家珍，基本上面试官会被你震惊到、吸引到。

最终，让面试官爱到“不能自己、口水直流”。offer，也就来了。

在面试之前，建议大家系统化的刷一波 5000页《[尼恩Java面试宝典PDF](#)》，里边有大量的大厂真题、面试难题、架构难题。很多小伙伴刷完后，吊打面试官，大厂横着走。

在刷题过程中，如果有啥问题，大家可以来找 40岁老架构师尼恩交流。

另外，如果没有面试机会，**可以找尼恩来改简历、做帮扶。**

遇到职业难题，找老架构取经，可以省去太多的折腾，省去太多的弯路。

尼恩指导了大量的小伙伴上岸，前段时间，**刚指导一个40岁+被裁小伙伴，拿到了一个年薪100W的offer。**

狠狠卷，实现“offer自由”很容易的，前段时间一个武汉的跟着尼恩卷了2年的小伙伴，在极度严寒/痛苦被裁的环境下，offer拿到手软，实现真正的“offer自由”。

技术自由圈 技术圣经系列

技术自由圈 7个 学习圣经 PDF

- 1 《SpringCloud Alibaba 学习圣经》
v1 版，发布日期：2023年03月15日 377页 PDF
- 2 《Docker 学习圣经》
v2 版，发布日期：2023年03月12日 145页 PDF
- 3 《Kubernetes 学习圣经》
v1 版，发布日期：2023年04月11日 530页 PDF
- 4 《响应式圣经》
v2 版，发布日期：2023年02月14日 114页 PDF
- 5 《缓存之王 Caffeine 红宝书》
v2 版，发布日期：2022年11月30日 175页 PDF
- 6 《队列之王 Disruptor 红宝书》
v1 版，发布日期：2022年10月05日 75页 PDF
- 7 《Prometheus+grafna 红宝书》
v1 版，发布日期：2022年10月08日 100页 PDF

技术自由圈 3个 高并发圣经 PDF

- 8 《Java高并发核心编程 卷1 加强版：NIO、Netty、Redis、ZooKeeper》
加强版，发布日期：2023年01月01日 609页 PDF
- 9 《Java高并发核心编程 卷2 加强版：多线程、锁、JMM、JUC、高并发设计模式》
加强版，发布日期：2022年11月14日 448页 PDF
- 10 《Java高并发核心编程 卷3 加强版：亿级用户Web应用架构与实战》
加强版，发布日期：2022年11月14日 481页 PDF

技术自由圈 面试圣经 40个 面试题 PDF

- 11 《尼恩 Java 面试宝典》40个专题
v60 版，发布日期：2023年03月11日 4000页 PDF

领取方式：技术自由圈 公众号



硬核推荐：尼恩Java硬核架构班

详情：<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



尼恩java 硬核架构班



已经发布

- ★ 《高性能RPC的基础实操之：从0到1开始IM撸一个IM》
- ★ 《分布式高性能RPC的基础实操之：千万级用户分布式IM实操- 含简历指导》
- ★ 《亿级用户超高并发秒杀实操- 含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，**2023夏招面试涨薪神器**
- ★ 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- ★ 《第1部曲：超级底层：葵花宝典（高性能秘籍）__架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023夏招面试涨薪大神器
- ★ 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- ★ 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- ★ 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- ★ 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- ★ 《架构师实操篇：redis cluster 工业级高可用实操》
- ★ 《架构师实操篇：100W级别QPS日志平台实操》
- ★ 《彻底穿透：skywalking 源码(代表链路跟踪)+Java agent+bytebuddy 探针》
- ★ 《超高并发场景100Wqps三级缓存组件原理和实操》
- ★ 《全链路异步超底层原理和实操：手写hystrix熔断+webflux+Lettuce+Dubbo》

规划中



左手大数据 (写入简历, 让简历 蓬荜生辉、金光闪闪)

HBASE + Flink + ElasticSearch 原理、架构、真刀实操



右手云原生 (写入简历, 让简历 蓬荜生辉、金光闪闪)

K8S + Devops + ServiceMesh 原理、架构、真刀实操

架构师实操篇: 基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架

架构师实操篇: go语言学习, 以及基于 go 手写 rpc 框架

架构师实操篇: 千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目

架构师实操篇: 工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目

特色

会员制

提供技术方向指导,
职业生涯指导, 少躺坑, 少弯路

简历指导

这个很重要,
对于挪窝涨薪来说

实操性

以上项目, 都是老架构师
在生产上实操过的项目

非水货

40岁老架构师, 不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

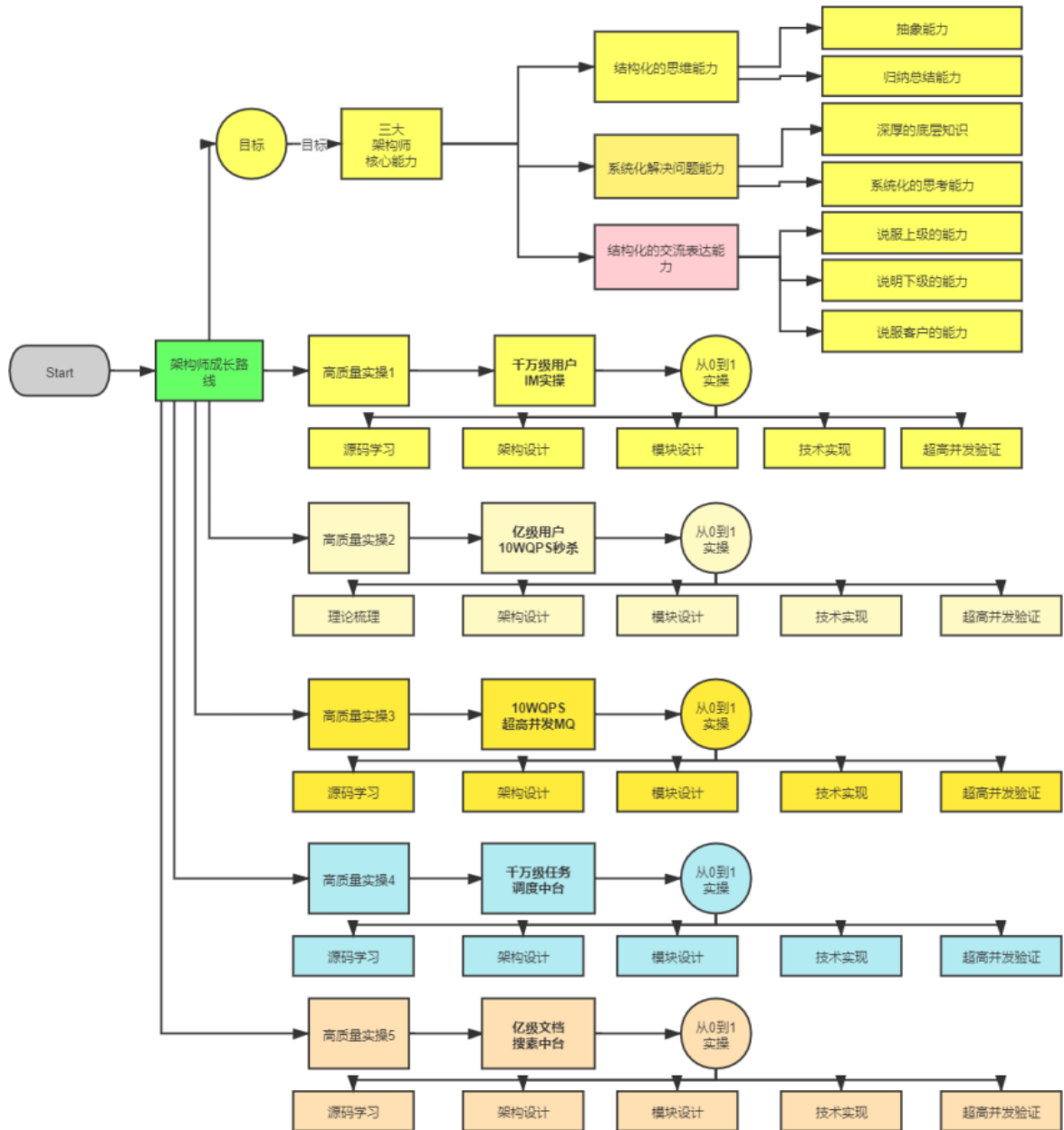
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



工业级 rocketmq 高可用底层原理和搭建实操，包含：高可用集群的搭建。

- 1、技术难题：RocketMQ 如何最大限度的保证消息不丢失的呢？RocketMQ 消息如何做到高可靠投递？
- 2、技术难题：基于消息的分布式事务，核心原理不理解
- 3、选型难题：kafka or rocketmq，该娶谁？

[illegible]

成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情：<http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

最新 最后发表 热门 精华

成功案例：[1057号卷王] 3年小伙拿到外企offer，薪酬涨了200%

1 卷王1号 超级版主 前天 17:41

成功案例：[645号卷王] 4年经验卷王逆袭，被毕业后，反涨24W

1 卷王1号 超级版主 2022-9-21

成功案例：[878号卷王] 小伙8年经验，年薪60W

1 卷王1号 超级版主 2022-8-13

年薪70W案例：通过尼恩的指导，小伙伴年薪从40W涨到70W

1 卷王1号 超级版主 2022-2-11

成功案例：[493号卷王] 5年小伙拿满意offer，就业寒冬季逆涨30%

1 卷王1号 超级版主 前天 17:43

成功案例：[250号卷王] 就业极寒时代，收offer 涨25%

1 卷王1号 超级版主 前天 17:38

成功案例：[612号卷王] 就业极寒时代，从外包到自研

1 卷王1号 超级版主 前天 17:15

成功案例：[913号卷王] 热烈祝贺6年经验卷王，年薪40W

1 卷王1号 超级版主 2022-9-21

成功案例：[959号卷王] 4年经验卷王，喜获百度、Boss直聘等N个优质offer，最高涨100%

1 卷王1号 超级版主 2022-9-21

成功案例：[529号卷王] 5年经验卷王喜收2大offer，最高涨5K

1 卷王1号 超级版主 2022-9-21

成功案例：[811号卷王] 热烈祝贺7年经验卷王，薪酬涨30%

1 卷王1号 超级版主 2022-9-21

成功案例：[287号卷王] 不惧大寒潮，卷王逆市收4 offer，涨30%，可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例：[1002号卷王] 5月份“被毕业”，改简历后，斩获顶级央企Offer，涨薪7000+

1 卷王1号 超级版主 2022-7-5

成功案例: [7号卷王] 热烈祝贺小伙伴涨薪120%

1 卷王1号 超级版主 2022-8-13

成功案例: [134号卷王] 大三小伙卷1年, 斩获顶级央企Offer, 成功逆袭

1 卷王1号 超级版主 2022-7-6

成功案例: [1008号卷王] 5年经验卷王收42W offer, 月涨8000, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [453号卷王] 非全日制 6年卷王喜提3 offer, 年薪30W, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [924号卷王] 6年卷王喜提4 offer, 最高涨薪9000, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [15号卷王] 4年卷王入职 微软, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-12

成功案例: [527号卷王] 4年卷王喜提2 offer, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-13

成功案例: [788号卷王] 3年卷王喜提优质Offer, 涨薪60%

1 卷王1号 超级版主 2022-5-11

成功案例: 热烈祝贺: 非全日制卷王, 喜提2个心仪offer, 面3家过2家

1 卷王1号 超级版主 2022-4-21

成功案例: [693号卷王] 二线城市6年卷王喜提4大优质Offer, 含央企offer, 最高薪酬35W

1 卷王1号 超级版主 2022-4-16

成功案例: [85号卷王] 双非2本小伙, 春招大捷, 喜提9个offer, 最高薪酬近30万

1 卷王1号 超级版主 2022-4-14

成功案例: [741号卷王] 卷王逆袭! 6年小伙从很少面试机会到搞定35K*14薪Offer

1 卷王1号 超级版主 2022-4-12

成功案例: [642号卷王] 热烈祝贺, 6年卷王喜提优质国企offer

1 卷王1号 超级版主 2022-4-7

成功案例: [796号卷王] 热烈祝贺, 36岁卷王喜提52万优质offer

1 卷王1号 超级版主 2022-3-25

❑ 成功案例: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

① 卷王1号 超级版主 2022-3-24

❑ 成功案例: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

① 卷王1号 超级版主 2022-3-21

❑ 成功案例: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

① 卷王1号 超级版主 2022-3-20

❑ 成功案例: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

① 卷王1号 超级版主 2022-3-16

❑ 简历案例: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 祝贺我圈两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

 尼恩 超级版主 2022-3-5

❑ 成功案例: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂Offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [449号卷王] 应届毕业卷王喜收 滴滴offer, 年薪33W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

① 卷王1号 超级版主 2022-2-10

❑ [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

① 卷王1号 超级版主 2022-2-10

简历优化后的成功涨薪案例（VIP含免费简历优化）

6年专科，2年翻4倍

2年从8K涨到35K

2021年从8K涨到22K

高并发 VIP76

老师，求助。

现在有两个满意的 offer，不知道怎么抉择。

一个是吉利，17k，大数据与 ai 部门。

另一个是一个平台，从零开始用 java 重写现在的项目，分布式架构，带团队，自己招人。22k，我觉得我说少了，我自己提的，然后今天发了 offer

呵呵，你太牛了

我也不好说

工资高的是个小公司，不到 50 人

感觉好事都被你占了

这一年半，真的谢谢您。

呵呵，相互交流，相互成长。

您写的书本，解决了我项目上很多问题。您在群里不厌其烦地告诉我们学习，也是我能坚持下来的重要因素，还有每次提问您都能解答疑惑，让我始终能戒骄戒躁。恩师，

**秘诀：
简历指导+ 狠狠卷**

2022年涨到35K

VIP76

解决了，限制 ip 频率。

谢谢老师

中午12:43

调整到了 35,加上这个月加班费，38

中午12:43

老师，我隔壁来了。

晚上8:23

大大的赞

老师你这路子是对的。我就跟着你学习思路和方法，还有教程走的。

我和你一样的兴奋和喜悦

记得咱们去年改简历的时候，还是 10k

这种提升，已经太令人震撼啦

是 8K...

20 年 4 月份转行，就一路跟着你学习





4年经验卷王逆袭 被毕业后，反涨24W

7月改简历

8月30日晒offer



小伙5月份"被毕业"，改简历后 斩获顶级央企Offer 涨薪7000+

5月29日改简历

7月5日晒offer



卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W

2月9日改简历

4月15日晒offer

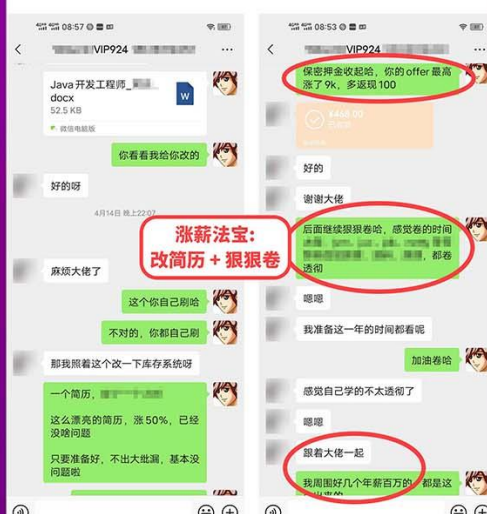


卷王逆袭成功案例

6年小伙喜提4个Offer 最高涨9k，年薪35W

4月14日改简历

5月17日晒offer



卷王逆袭成功案例

5年经验小伙收2个offer 最高涨薪8k，年薪42W

5月9日改简历

5月30日晒offer

秘诀:
简历指导+ 狠卷3高

以此为样
大家狠狠卷
打造最卷IT社群

卷王逆袭成功案例

非全日制 6年经验卷王 喜提3个Offer，年包30W

5月9日改简历

5月18日晒offer

面试法宝:
改简历+ 狠狠卷

卷王逆袭成功案例

寒五冻六之际卷王大逆袭 收3大offer，涨30%

5月17日改简历

5月27日晒offer

秘诀:
简历指导+ 狠卷3高

卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历

5月12日晒offer

涨薪法宝:
改简历+ 狠狠卷

4年小伙喜收百度、Boss直聘等N个顶级Offer 最高涨幅100%

6月27日改简历 9月19日晒offer

**秘诀：
改简历 + 狠狠卷**

有个offer选择问题

boss直聘和小满之间，boss那边给的offer，工资比小满高，但是小满那边给的offer，福利比boss高，不知道该怎么选，如果你应该怎么选呀。

还有其他很多offer的，还有一个小满的，总体上看boss和小满之间选一个

了

boss比小满年包多7w以上，我这涨幅都接近百分之百了

太牛啦

按照你那个思路结合上次指导的内容改了几个点，发出去确实大上了很多，加上一些指标的说明确实会让人眼前一亮的感觉。

明天难时候有没了我和我看下哈，一是看两个点我实在不知道该怎么改了，二是我在推荐表达这方面确实不行，还得让你帮忙把把关。

卷王逆袭成功案例 4年卷王入收2个offer，涨50%

3月23日改简历 5月12日晒offer

offer决策图

n+1 电商erp，部门成熟，人数多，加班少。

n+2 做业务中台 内部系统，新部门，人数少，加班多。

又搞到个offer

能搞定两个offer，不尴尬

现在很多小伙伴面试机会都没有

涨了多少呀

地点在哪里

涨50%的样子

忘记你工作几年啦，大概工作几年啦

**涨薪法宝：
改简历 + 狠狠卷**

这个不用加粗

都是一样，加粗了反而没有效果

下边请继续，有不懂的问我

小伙大三暑期很焦虑 跟着尼恩卷一年 校招斩获顶级央企Offer

去年5月19日加入VIP群 今年7月5日晒offer

**秘诀：
狠狠卷书+视频**

邀请你加入群聊

尼恩老师

我校招去华润电力控股有限公司了

跟着你卷了一年 大学顺便拿了几个国奖

不错不错，这是央企

放心中

这太牛啦

其实拿到手的也就一个a类国一

尼恩大佬 大三的暑期实习找不到，现在准备秋招应该没关系吧

看身边总是很焦虑，自己算法这一块卡住

网盘里边有算法视频

去刷一刷吧

秋招来得及

谢谢大佬 不过经过几个月练习，看您写的书比之前轻松多了

趁着还是学生这段时间，慢慢把知识吃透

嗯，我的书，比较深

期待大佬的下一本书，已经迫不及待去学习了

小伙高中学历 薪酬涨120%

5月6日改简历 7月22日晒offer

你这块估计要送你668

668.00

模块的开发工作，就这个三个哈

哈哈，不用了卡吧，您真的厉害了，光今天晚上辅导就值几千了

老弟很感谢您

还有很多其他的模块

面试官提问

就说是其他人做的

嗯，好

**秘诀：
改简历 + 狠狠卷**

之前你的工资是多少来的

翻了一翻

我得送你多少奖金来的

不知道，原价给老弟就行了

668.00

哈哈，您真配了一笔奖金

咱们得说这话呀

老弟拿着您的那个高开发改造亮点，所向披靡。

ok

从从到尾给面试官讲明白

后面继续狠狠卷哈

卷王逆袭成功案例

非全日制卷王 面试3家
收2个offer 涨薪30%

4月13日改简历

4月21日晒offer

5年卷王喜收2大Offer

最高涨5K

5月19日改简历

9月13日晒offer

面试法宝:
改简历 + 面试题

卷王逆袭成功案例

3年经验卷王，涨60%

4月16日改简历

5月11日晒offer

卷王逆袭成功案例

双非二本小伙春招大翻身
喜提9大offer

2月22日改简历

4月13日晒offer

面试法宝:
改简历 + IM实操

9大offer 最高年薪30万

序号	公司	部门	岗位	薪资结构	总包
1	公司	数据数字化产品部	java后端开发	18.5k+14.5k+5k+200k/每月+500k/期权	>30w
2	公司	交易研发部	java后端开发	16k+14k	22.4w
3	公司	待定	java游戏开发	15k+15k+餐补+100k/每月	>22.5w
4	公司	待定	java后端开发	11k+13k	14.2w
5	公司	待定	java后端开发	14k	>16.8w
6	公司	待定	java后端开发	9k	9k
7	公司	待定	java后端开发	9k+包房租+报销津贴	10k+餐补+房补
8	公司	待定	java后端开发	10k+餐补+房补	17w
9	公司	待定	java后端开发	10k+餐补+房补	17w

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>